



Ollscoil Chathair  
Bhaile Átha Cliath  
Dublin City University

# Cellular Automata - Simulations (labs)

CSC1111  
Building Complex Computational Models

Marija Bezbradica

## Monte Carlo - reminder

When you work on the CA tasks, make sure to carry out **multiple independent simulation** runs for each value of parameters (e.g.  $p$ ) and **average** the results to obtain statistically more reliable results.

Simulation methods based on random sampling are generally called **Monte Carlo simulations**.

In Monte Carlo simulations, we conduct many **replications of independent simulation runs** for the same experimental setting and measure the outcome variables from each run to obtain a **statistical distribution of the measurements**.

Using this distribution (e.g., by calculating its average value) will help enhance the **accuracy** and **reliability** of the experimental results.

### Example 8.3. (Sayama book)

We want to implement is a binary CA model with the droplet rule. Its state-transition function can be understood as a model of **panic propagation among individuals** sitting in a gym after a fire alarm goes off.

Here is the rule (which uses the Moore neighbourhoods):

- A normal individual will get panicky if he or she is surrounded by four or more panicky individuals.
- A panicky individual will remain panicky if he or she is surrounded by three or more panicky individuals. Otherwise he or she will revert back to normal.

Note that this rule can be further simplified to the following single rule:

- If there are four or more panicky individuals within the neighborhood, the central cell will become panicky; otherwise it will become normal.

Here are other model assumptions:

- Space: 2-D,  $n \times n$  ( $n = 100$  for the time being)
- Boundary condition: periodic
- Initial condition: Random (panicky individuals with probability  $p$ ; normal ones with probability  $1 - p$ )

### **Example 8.3.** (Sayama book)

We will use *pycxsimulator.py* for dynamic CA simulations (introduced in week 6). First, we need to design and implement the three essential components:

**1. initialization** - a regular 2D grid, it is natural to use Python's *array* data structure. We can initialize it with randomly assigned states with probability  $p$ . Prepare two such arrays, one for the current time step and the other for the next time step, in order to avoid any unwanted conflict during the state updating process. So, an example of how to implement the initialization part:

$n = 100$  # size of space:  $n \times n$

$p = 0.24$  # probability of initially panicky individuals

```
def initialize():
```

```
    global config, nextconfig # config, rnd assigned values, nextconfig, not rnd assigned values
```

```
    config = zeros([n, n])
```

```
    for x in range(n):
```

```
        for y in range(n):
```

```
            config[x, y] = 1 if random() < p else 0
```

```
    nextconfig = zeros([n, n])
```

## Example 8.3. - continued

### 2. observation

- pylab has a built-in function *imshow* that is perfect for our purpose to visualize the content of an array.
- *cmap* option in *imshow* is to specify the color scheme used in the plot. Without it, pylab uses dark blue and red for binary values by default, which are rather hard to see.

```
def observe():  
    global config, nextconfig  
    cla()  
    imshow(config, vmin = 0, vmax = 1, cmap = cm.binary)
```

## Example 8.3. - continued

### 3. updating –

- Algorithmic thinking. The state-transition function needs to count the number of panicky individuals within a neighborhood.
- Moore neighbourhood around a position (x; y) can be written as: (so, we can use nested for loops):

$$\{(x', y') \mid x - 1 \leq x' \leq x + 1, y - 1 \leq y' \leq y + 1\}.$$

- The expression (...) % n means that the value inside the parentheses is contained inside the [0; n- 1] range by the mod operator (%). A useful coding technique to implement periodic boundary conditions in a very simple manner.

**\*Full code is uploaded in Week 8 under '8.2. ca-panic.py'**

```
def update():
    global config, nextconfig
    for x in range(n):
        for y in range(n):
            count = 0
            for dx in [-1, 0, 1]:
                for dy in [-1, 0, 1]:
                    count += config[(x + dx) % n, (y + dy) % n]
            nextconfig[x, y] = 1 if count >= 4 else 0
    config, nextconfig = nextconfig, config
```

### Example 8.4. (Sayama book)

- Modify Code **Example 8.3** to implement a simulator of the **Game of Life CA** (on Loop: '8.2. ca-panic.py').
- Simulate the dynamics from a random initial configuration. Measure the density of state 1's in the configuration at each time step, and plot how the density changes over time.

This can be done by creating an empty list in the initialize function, and then making the measurement and appending the result to the list in the observe function.

The results stored in the list can be plotted manually after the simulation, or they could be plotted next to the visualization using pylab's subplot function during the simulation.

### Example 8.5. (Sayama book)

Modify Code **Example 8.3** ('8.2. ca-panic.py') to implement a simulator of the **majority rule** CA in a two-dimensional space.

Then answer the following questions by conducting simulations:

1. What happens if you change the ratio of binary states in the initial condition?
2. What happens if you increase the radius of the neighborhoods?
3. What happens if you increase the number of states?



### Example 8.6. (Sayama book)

- Implement an **interactive parameter setter** for  $p$  in **Example 8.3** ('8.2. ca-panic.py')
- Conduct systematic simulations with varying  $p$ , and identify its critical value below which isolated clusters are formed but above which the whole space is filled with panic.

### Example 8.7. (Sayama book)

Implement a CA model of **epidemic/forest fire systems** in Python.

Then try the following:

- Compare the final results for different values of  $p$ .
- Plot the total burned area as a function of  $p$ .
- Plot the time until the fire stops spreading as a function of  $p$ .
- **Reminder: Some solutions and other cool examples are available on our toolkit page on <https://github.com/hsayama/PyCX>**

- Upload all your lab results for week 8 under '[Week 8: Cellular Automata](#)'